

Sanipatitas — HC Veterinaria

Arquitectura Orientada al Servicio

Alcántara Paico, Luis Sebastián

Contreras Delgado, Jorge Renzo Paolo

Huamán Domador, Luis Alberto

Ramos Yampufe, Martin Alexander

Ruiz Cardoza, Richard

2026-07-20

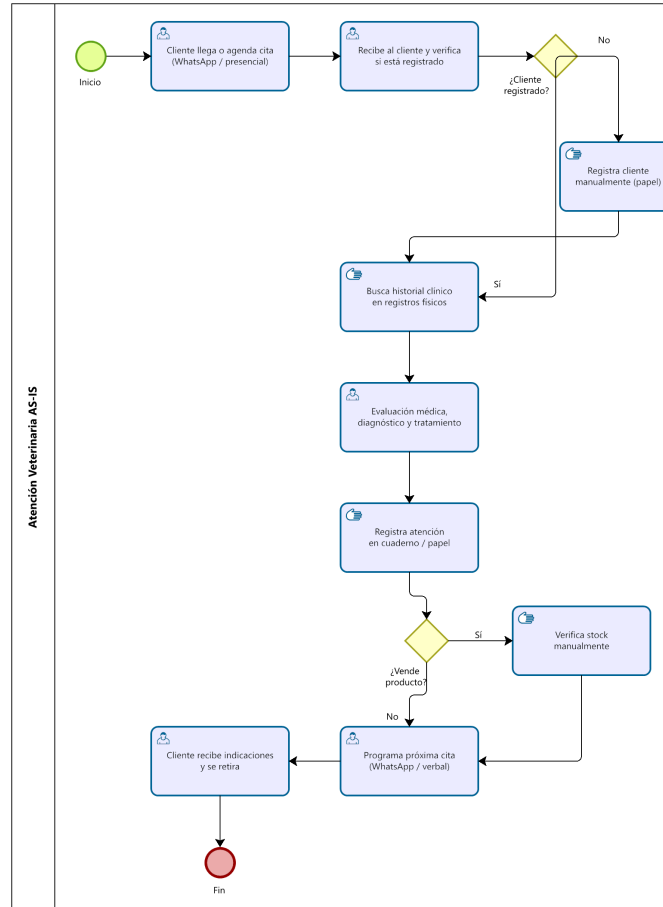
Universidad Tecnológica del Perú

1. Contexto actual

1. Contexto actual

La clínica opera sin sistema digital. Todo se registra en papel, cuadernos y WhatsApp.

1. Contexto actual



2. Servicios del backend

2. Servicios del backend

Servicio	Interfaz REST	Implementación
Auth Service	POST /sign-in/email + GET /jwks + admin CRUD	Bun + Elysia + Better Auth
Patient Service	CRUD /patient + /client + /species + /breed	Java 25 + Quarkus 3.37.0
Appointment Service	CRUD /appointment + SSE events	Java 25 + Quarkus 3.37.0
EHR Service	CRUD /clinical/* (condition, observation, immunization)	Java 25 + Quarkus 3.37.0
Inventory Service	CRUD /inventory/* (product, stock, supplier)	Java 25 + Quarkus 3.37.0
Payment Service	CRUD /billing/* (billing, item, payment)	Java 25 + Quarkus 3.37.0

3. Frontend

3. Frontend

Astro 6 + React 19, Tailwind CSS v4, TanStack Router. Cliente API generado desde OpenAPI.

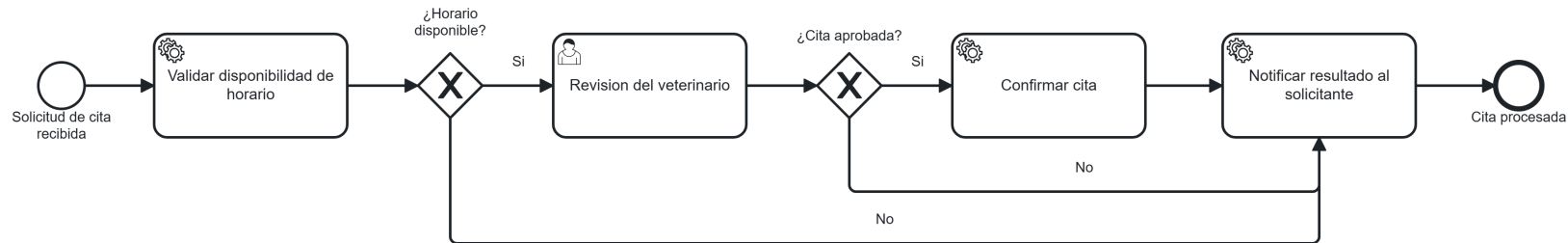
Capa	Tecnología
Routing	TanStack Router
Estilos	Tailwind CSS v4 + @base-ui/react
Estado	TanStack React Query + nanostores
Cliente API	@hey-api/openapi-ts desde OpenAPI
Distribución	PWA + Tauri (escritorio)

4. Flujo de atención integral

4. Flujo de atención integral

El frontend orquesta seis servicios en una secuencia definida:

1. Escaneo de QR de la mascota → Patient Service
2. Verificación de cita → Appointment Service
3. Actualización de historial clínico → EHR Service
4. Prescripción de receta → EHR Service (prescription)
5. Facturación y descuento de stock → Payment + Inventory Service
6. Confirmación y QR actualizado



5. Identificación por QR

5. Identificación por QR

Cada mascota tiene un código QR único con el protocolo sanipatitas://patient/{uuid}.

Escaneo del código QR → Extracción del UUID → Consulta GET /api/patient/{uuid} →
Ficha del paciente en pantalla

El QR se genera al registrar la mascota. El propietario recibe una etiqueta impresa para colocarla en el collar.

6. API Gateway — HAProxy

6. API Gateway — HAProxy

Punto de entrada único que enruta según el prefijo de la ruta:

<code>/api/auth</code>	<code>-> auth:3000</code>
<code>/api/patient</code>	<code>-> patient:8080</code>
<code>/api/appointment</code>	<code>-> appointment:8081</code>
<code>/api/clinical</code>	<code>-> ehr:8082</code>
<code>/api/inventory</code>	<code>-> inventory:8083</code>
<code>/api/billing</code>	<code>-> payment:8084</code>
<code>/*</code>	<code>-> frontend:1420</code>

Cada servicio valida el JWT de forma autónoma consultando el endpoint JWKS del Auth Service.

7. Autenticación JWT

7. Autenticación JWT

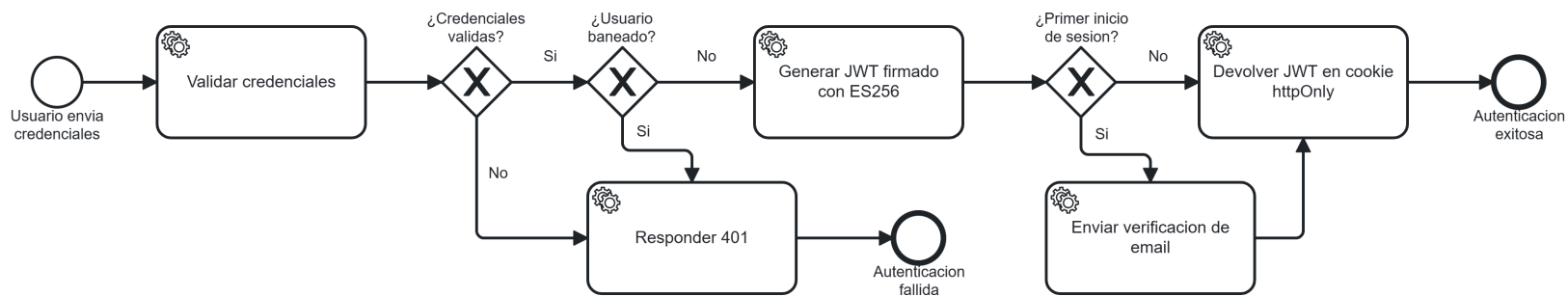


Figure 1: Flujo de autenticación y autorización

JWT firmado con ES256 (curva elíptica P-256). Sesiones en Redis. Contraseñas con bcrypt. Renovación automática de tokens.

8. Roles del sistema

8. Roles del sistema

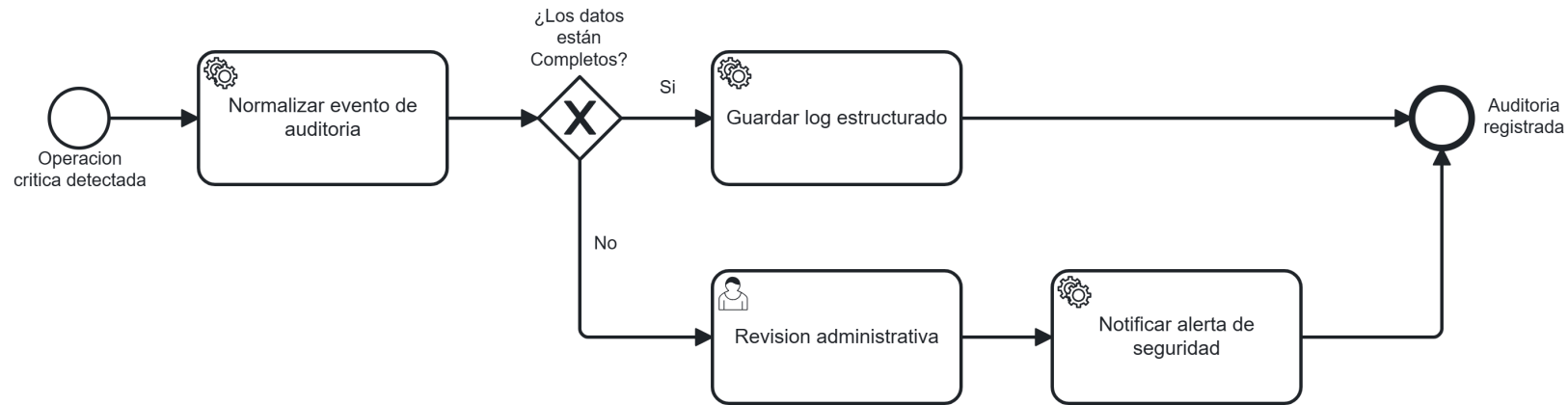
Rol	Permisos	Módulos
Administrador	CRUD completo	Usuarios, pacientes, clientes, citas
Veterinario	Lectura, creación, actualización	Historial médico, citas
Trabajador	Lectura, creación, actualización	Clientes, pacientes, citas

@RolesAllowed en controladores. DELETE solo para admin. Token inválido → 401. Sin permisos → 403.

9. Modelado BPMN

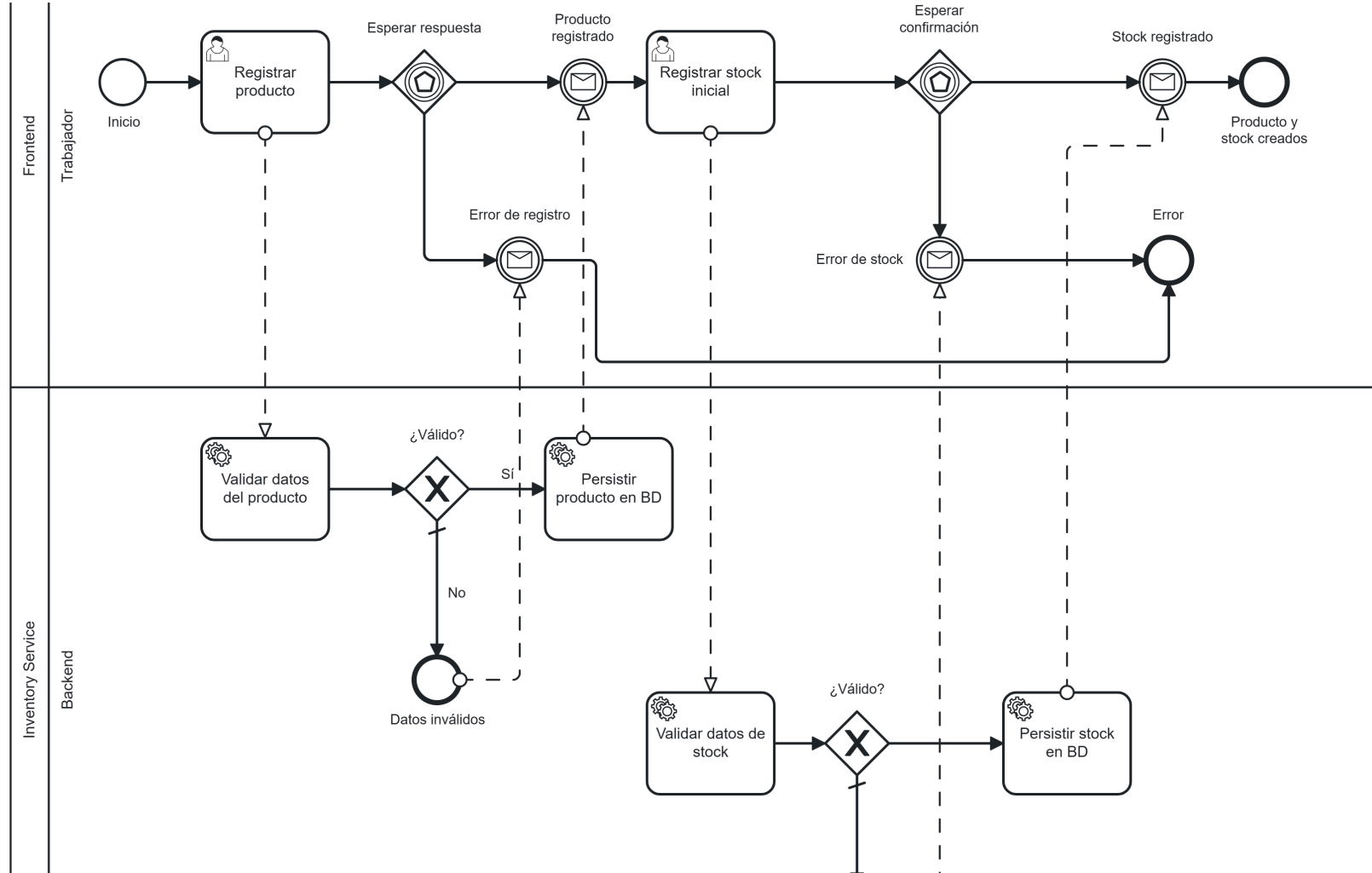
9. Modelado BPMN

Procesos modelados con Camunda 8 y BPMN 2.0.



Cinco procesos: auditoría, aprobación de citas, gestión de stock, registro de paciente, autenticación.

9. Modelado BPMN



10. Auditoría del sistema

10. Auditoría del sistema

Tres capas complementarias para dejar evidencia de toda operación crítica.

- Logs en tiempo real: AuditLogFilter (Quarkus) + auditMiddleware (Elysia) emiten logs JSON planos
- Hibernate Envers: Tablas `_aud` con snapshots históricos de cada entidad de negocio
- Centralización: Logs enviados directamente a Loki mediante HTTP. Dashboards en Grafana

Trazabilidad extremo a extremo mediante `correlationId` propagado entre servicios.

11. Conclusiones

11. Conclusiones

- Seis servicios backend independientes (Auth, Patient, Appointment, EHR, Inventory, Payment)
- Frontend Astro 6 + React 19 con cliente API generado desde OpenAPI
- Autenticación JWT ES256 con validación descentralizada mediante JWKS
- Auditoría en tres capas con trazabilidad por `correlationId`
- Códigos QR para identificación rápida de mascotas
- Despliegue con Docker + k3s

Gracias